# Implementation of a Remote Hardware-in-the-Loop System for FPGA Configuration and Control via Heterogeneous Network Protocols

Rewan Elbebawy, Farah Haitham, Fady Salah, Safa Gamal, Yassin Ahmed, Nour Ahmed
*Faculty of Information Engineering and Technology, Electronics Department*
German University in Cairo

***Abstract*—This paper details the design and implementation of a remote Hardware-in-the-Loop (HIL) testing environment, enabling seamless configuration and control of a Basys3 Field Programmable Gate Array (FPGA). Utilizing a Raspberry Pi as an intermediary controller, the system bridges a host PC to the FPGA over both local and Virtual Private Network (VPN) architectures. A heterogeneous protocol approach was adopted: TCP sockets for reliable bitstream transmission, MQTT for high-speed control messaging, and SPI for synchronous hardware-level communication. The paper discusses the integration of the openFPGALoader library for physical programming via JTAG, the precise multi-threaded architecture required for concurrent I/O operations, network constraints addressed via tailored packet chunking over Tailscale VPN, and future avenues for multi-user concurrency and scheduling.**

***Index Terms*—FPGA, Hardware-in-the-Loop, MQTT, SPI, TCP, VPN, Tailscale, openFPGALoader, Multi-threading**

## I. Introduction

The ability to remotely program, test, and receive feedback from hardware accelerates development cycles and enables flexible resource sharing. This project presents a robust HIL architecture that allows a user to remotely upload a synthesized `.bit` file to an FPGA and interface with it in real-time. The system requires zero physical interaction at the hardware site beyond connecting the Raspberry Pi to a power source. Upon booting the host PC and initiating the local web server, the user gains complete control over the FPGA testing pipeline via a dedicated graphical user interface (GUI).

## II. System Architecture

The system is divided into three primary nodes: the Host PC, the Raspberry Pi Controller, and the FPGA.

### A. Host PC Environment

The host PC acts as the user-facing terminal. The operation is initialized by executing the `serverPC.py` script, which establishes a local web server accessible via `http://127.0.0.1:5002`. From this interface, the user can upload the required `.bit` file and send operational commands. The host environment utilizes modular client scripts, including `tcp_client.py` for file transfer and `mqtt_client.py` for telemetry.

### B. Intermediary Controller (Raspberry Pi)

The Raspberry Pi serves as the core bridging unit. It runs a suite of server-side scripts, notably `tcp_serverPi.py` and `mqtt_clientPi.py`, managed by a central `serverPi.py` routine.

**1) FPGA Programming Interface:** The physical connection between the Pi and the FPGA utilizes a standard USB Type-A to Micro-USB cable. It is crucial to note that this specific interface is strictly dedicated to the initial programming phase of the FPGA. The Basys3 board features an onboard FTDI chip (FT2232HQ) that bridges USB to the FPGA's native JTAG interface. By utilizing the **openFPGALoader** library, the Raspberry Pi can act as the USB host, sending the `.bit` file directly through this JTAG bridge to flash the FPGA's memory. *openFPGALoader* was deliberately selected for its lightweight, open-source nature, allowing for rapid bitstream deployment via Linux-based ARM architectures without the need for massive, proprietary toolchains like Xilinx Vivado.

The design choice to restrict the USB connection *solely* to programming is rooted in the latency requirements of the HIL system. USB is a host-polled, packet-based protocol, which introduces unpredictable latency and high software overhead. It is entirely unsuitable for the strict, deterministic, high-speed data exchange required during active testing. Therefore, once the programming is complete, the system immediately shifts to direct GPIO hardware pins (SPI) for active telemetry.

**2) Multi-threading Implementation:** A critical architectural feature of the Raspberry Pi controller is its multi-threading implementation. The system must simultaneously manage three heavily I/O-bound communication schemes: TCP (listening for heavy file transfers), MQTT (handling asynchronous control signals), and SPI (driving continuous hardware I/O). Running these sequentially in a single loop would cause catastrophic bottlenecking, where a blocking network call (like waiting for a TCP packet) would halt all hardware interaction.

To solve this, the software architecture leverages isolated execution threads running concurrently:

- **The TCP Thread:** Dedicated entirely to the `socket.accept()` blocking call, allowing the Pi to passively listen for incoming `.bit` files without freezing the main control logic.

- **The MQTT Thread:** Utilizes an asynchronous event loop that triggers specific callback functions whenever a new control command (operation/operands) is published by the host PC.
- **The SPI/Hardware Thread:** A continuous, high-speed loop dedicated to clocking data into and out of the FPGA via the GPIO pins.

To ensure stability and prevent race conditions—such as the SPI thread attempting to read an operand at the exact millisecond the MQTT thread is updating it—the system utilizes thread synchronization primitives (e.g., Mutex locks or thread-safe queues). This guarantees mutually exclusive access to shared variables, allowing the Raspberry Pi to concurrently receive network commands and immediately pipeline them to the FPGA with microsecond precision.

## III. Communication Protocols

To optimize performance and reliability, different communication protocols were mapped to specific data types based on their inherent characteristics.

The decision to implement a heterogeneous networking approach—specifically utilizing both TCP and MQTT rather than standardizing on a single protocol—was driven by the distinct payload requirements of the system. Relying solely on MQTT would risk file corruption during large transfers, while relying solely on TCP would introduce unacceptable latency in real-time control.

### *A. Transmission Control Protocol (TCP) for Configuration*

The transfer of the `.bit` configuration file is handled via a dedicated TCP socket connection. TCP is a heavy, connection-oriented protocol that guarantees ordered, error-free delivery. This strict error-checking is an absolute necessity for transmitting compiled binary files, where a single dropped or out-of-order byte results in a corrupted hardware configuration.

### *B. Message Queuing Telemetry Transport (MQTT) for Telemetry*

While TCP is ideal for bulk transfer, utilizing it for continuous, real-time control telemetry would introduce unnecessary overhead. Therefore, control messages—specifically the operation codes and operands sent from the host PC—are transmitted utilizing the **HiveMQ public broker**. MQTT operates on a lightweight, publish-subscribe model ideal for high-frequency, low-latency control messages, providing an optimal balance when paired with TCP.

### *C. Serial Peripheral Interface (SPI) for Hardware I/O*

Communication between the Raspberry Pi and the FPGA is handled via SPI at a clock speed of 1 MHz. SPI was selected because it is highly efficient and noticeably easier to implement for hardware-level data exchange. Because the clock signal is shared between the master (Pi) and slave (FPGA), it inherently avoids the synchronization problems and timing overhead associated with asynchronous protocols like UART, making it ideal for rapid control and feedback loops.

## IV. Practical Challenges and System Modifications

During the development and testing phases, several practical challenges necessitated architectural modifications.

### *A. Port Conflict Resolution*

Initially, both the primary web server and the TCP file-transfer socket were configured to run on the same network port, leading to immediate binding conflicts and process crashes. This was resolved by explicitly separating the network traffic: the web server was bound to Port 5002, while the TCP socket connection was isolated on Port 5001.

### *B. Cross-Network VPN Implementation and MTU Constraints*

A core requirement of the project was that the host PC and the Raspberry Pi must be able to operate on entirely different networks. While the TCP protocol functioned flawlessly when both devices were on the same local Wi-Fi, TCP inherently requires devices to be on the same network to establish a direct connection.

To solve this issue and bridge the two different networks, **Tailscale** was implemented. Tailscale created a secure VPN tunnel that successfully assigned and maintained specific, static IP addresses for both the Raspberry Pi and the Host PC, regardless of their physical network locations. (This requires the host PC to download Tailscale and log into the shared user account).

However, a significant issue arose regarding the Maximum Transmission Unit (MTU) size. The Tailscale virtual tunnel possesses a restricted MTU size of 1280 Bytes. The initial TCP packet chunk size for the `.bit` file transfer was configured to 4096 Bytes. This mismatch caused severe packet fragmentation and connection timeouts over the VPN. To resolve this, the TCP payload chunk size was aggressively reduced to 1024 Bytes, ensuring it fit comfortably within the VPN tunnel's payload limits without fragmentation.

## V. Limitations and Future Work

While the current implementation successfully demonstrates a highly functional HIL system, there are specific limitations regarding its scalability.

Currently, the system's GUI and underlying data parsing logic are specialized for an Arithmetic Logic Shift Unit (ALSU) application. Future iterations will focus on generalizing the interface and data handling to support a wider array of FPGA applications and dynamic feedback signals.

Furthermore, expanding the system to allow ubiquitous access from any network introduces complex concurrency challenges. If multiple users attempt to access the remote FPGA simultaneously, it will result in resource contention and unpredictable hardware states. Future advancements will require the implementation of robust thread-blocking mechanisms to ensure mutually exclusive access to the FPGA. This will be paired with a scheduling or polling system to notify queued users with a "System Busy" status, dynamically alerting them when the hardware is free for their specific test bench.

## VI. Conclusion

This project successfully implemented a multi-threaded, remote HIL testing pipeline. By strategically deploying TCP for reliable bulk transfer alongside Tailscale for cross-network IP management, HiveMQ MQTT for low-latency control, and SPI for highly efficient, synchronous hardware interfacing, the system achieves a robust control loop. The integration of the openFPGALoader library and vital MTU chunk-size modifications ensures the system remains reliable across disparate network topologies, laying a strong foundation for future multi-user hardware sharing architectures.